

Web Development

Topic 1: HTML

1. Question: What is HTML, and why is it used in web development?

Answer: HTML stands for HyperText Markup Language. It is used to create and structure web pages.

Simple Explanation: HTML tells the browser how to display headings, paragraphs, images, links, and other webpage elements.

Example:

```
<h1>Welcome to Skill Sense</h1>  
<p>Learn and improve your skills.</p>
```

2. Question: What is the basic structure of an HTML document?

Answer: An HTML document contains the DOCTYPE, html, head, title, and body elements.

Simple Explanation: These elements organize a webpage and help the browser understand how to display its content.

Example:

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>My Website</title>  
</head>  
<body>  
  <h1>Hello World</h1>  
</body>  
</html>
```

3. Question: What is the purpose of the DOCTYPE declaration in HTML?

Answer: The DOCTYPE declaration tells the browser to render the page using modern HTML standards.

Simple Explanation: It helps the browser display the webpage in standards mode instead of quirks mode.

Example:

```
<!DOCTYPE html>
```

4. Question: What is the difference between the head and body elements?

Answer: The head contains metadata and resource references, while the body contains the visible webpage content.

Simple Explanation: The title, stylesheet links, and metadata usually go inside head. Headings, images, and paragraphs go inside body.

Example:

```
<head>
  <title>Skill Sense</title>
</head>
<body>
  <h1>Welcome</h1>
</body>
```

5. Question: What are HTML tags and elements?

Answer: Tags are markup instructions written inside angle brackets. An element consists of its tags and the content between them, when applicable.

Simple Explanation: Tags define how content is structured. Elements are the complete building blocks of an HTML page.

Example:

```
<p>This is a paragraph.</p>
```

6. Question: What are HTML attributes?

Answer: HTML attributes provide additional information or configuration for HTML elements.

Simple Explanation: Attributes are commonly written in the opening tag and often use a name-value pair.

Example:

```

```

7. Question: What is the difference between block-level and inline elements?

Answer: Block-level elements normally begin on a new line, while inline elements flow within the surrounding text.

Simple Explanation: A heading or paragraph generally occupies its own line. A link or emphasis element usually stays within a line of text.

Example:

```
<div>Block element</div>
```

`<span>Inline element</span>`

8. Question: How do you create headings in HTML?

Answer: HTML provides six heading levels, from h1 to h6.

Simple Explanation: These headings organize content into a hierarchy, with h1 representing the highest level.

Example:

```
<h1>Main Heading</h1>
```

```
<h2>Subheading</h2>
```

```
<h3>Section Heading</h3>
```

9. Question: How do you create hyperlinks in HTML?

Answer: The anchor element, a, creates hyperlinks using the href attribute.

Simple Explanation: A hyperlink lets users navigate to another page, website, or location.

Example:

```
<a href="https://example.com">Visit Website</a>
```

10. Question: How do you insert an image into an HTML page?

Answer: The img element displays an image using the src attribute.

Simple Explanation: The src specifies the image location, while alt provides alternative text if the image cannot be displayed or is accessed through assistive technology.

Example:

```

```

11. Question: What is semantic HTML?

Answer: Semantic HTML uses elements that describe the meaning and purpose of their content.

Simple Explanation: Elements such as header, nav, main, article, and footer make webpage structure easier to understand.

Example:

```
<header>Skill Sense</header>
```

```
<main>
```

```
  <article>Learning Content</article>
```

```
</main>
```

<footer>Copyright 2026</footer>

12. Question: What is the difference between ordered and unordered lists?

Answer: An ordered list uses ol to display numbered items, while an unordered list uses ul to display bulleted items.

Simple Explanation: Use numbered lists when sequence matters and bulleted lists when the order is not important.

Example:

```
<ol>
```

```
  <li>Learn HTML</li>
```

```
  <li>Learn CSS</li>
```

```
</ol>
```

```
<ul>
```

```
  <li>Programming</li>
```

```
  <li>Database</li>
```

```
</ul>
```

13. Question: What is the purpose of HTML comments?

Answer: HTML comments allow developers to add notes that are not displayed as webpage content.

Simple Explanation: Comments help explain sections of code and make projects easier to maintain.

Example:

```
<!-- Main navigation section -->
```

```
<nav>
```

```
  <a href="index.html">Home</a>
```

```
</nav>
```

Topic 2: CSS

14. Question: What is CSS, and why is it used?

Answer: CSS stands for Cascading Style Sheets. It controls the appearance and layout of HTML elements.

Simple Explanation: CSS makes webpages attractive by styling colors, fonts, backgrounds, spacing, and layouts.

Example:

```
h1 {
```

```
color: purple;
font-size: 32px;
}
```

15. Question: What are the different ways to apply CSS to HTML?

Answer: CSS can be applied using inline, internal, and external styles.

Simple Explanation: Inline CSS styles one element directly. Internal CSS is written inside a style element, while external CSS is stored in a separate file.

Example:

```
<link rel="stylesheet" href="style.css">
```

16. Question: What is a CSS selector?

Answer: A CSS selector identifies the HTML elements to which styles should be applied.

Simple Explanation: Selectors allow developers to style elements by tag name, class, ID, or other conditions.

Example:

```
p {
  color: blue;
}
```

17. Question: What is the difference between class and ID selectors?

Answer: A class can be reused on multiple elements, while an ID should uniquely identify an element within a page.

Simple Explanation: Use classes for reusable styling and IDs for unique elements.

Example:

```
<p class="description">Welcome</p>
<h1 id="main-title">Skill Sense</h1>
.description {
  color: gray;
}
```

```
#main-title {
  color: purple;
}
```

```
}
```

18. Question: What is the CSS box model?

Answer: The CSS box model consists of content, padding, border, and margin.

Simple Explanation: It explains how an element's content and surrounding spaces contribute to its size and layout.

Example:

```
.card {  
  width: 200px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 10px;  
}
```

19. Question: What is the difference between margin and padding?

Answer: Margin creates space outside an element's border, while padding creates space between its content and border.

Simple Explanation: Margin separates elements from each other. Padding creates breathing room inside an element.

Example:

```
.card {  
  margin: 20px;  
  padding: 15px;  
}
```

20. Question: How do you change text color and background color using CSS?

Answer: The color property changes text color, and background-color changes an element's background color.

Simple Explanation: These properties help establish a website's visual style and improve readability.

Example:

```
body {  
  background-color: #f0f0ff;  
  color: #222222;  
}
```

21. Question: What is CSS Flexbox?

Answer: Flexbox is a CSS layout system used to arrange elements in a row or column.

Simple Explanation: It helps align items, distribute available space, and build flexible layouts.

Example:

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

22. Question: What is CSS Grid?

Answer: CSS Grid is a layout system used to arrange elements in rows and columns.

Simple Explanation: Grid is useful for designing structured layouts such as dashboards, galleries, and card collections.

Example:

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
}
```

23. Question: What is the purpose of the CSS position property?

Answer: The position property controls how an element is positioned within a webpage.

Simple Explanation: Common values include static, relative, absolute, fixed, and sticky.

Example:

```
.header {  
  position: sticky;  
  top: 0;  
}
```

24. Question: What are CSS pseudo-classes?

Answer: Pseudo-classes style elements when they are in a particular state or position.

Simple Explanation: They can change the appearance of buttons when users hover over them or inputs when they receive focus.

Example:

```
button:hover {  
    background-color: purple;  
    color: white;  
}
```

25. Question: What are CSS transitions?

Answer: CSS transitions allow property changes to occur gradually over a specified duration.

Simple Explanation: They make interactions smoother instead of changing styles instantly.

Example:

```
button {  
    transition: background-color 0.3s ease;  
}
```

26. Question: What is the purpose of the z-index property?

Answer: The z-index property controls the stacking order of positioned elements and certain other layout elements.

Simple Explanation: It helps determine which overlapping element appears in front of another.

Example:

```
.modal {  
    position: fixed;  
    z-index: 1000;  
}
```

Topic 3: JavaScript

27. Question: What is JavaScript, and why is it used in web development?

Answer: JavaScript is a programming language used to add interactivity and dynamic behavior to webpages.

Simple Explanation: It can respond to clicks, validate forms, update content, and communicate with servers.

Example:

```
alert("Welcome to Skill Sense!");
```

28. Question: What is the difference between var, let, and const?

Answer: They declare variables, but differ in scope and reassignment rules.

Simple Explanation: let allows reassignment within its scope. const prevents reassignment of the binding. var has function scope and permits redeclaration in the same scope.

Example:

```
let score = 80;  
score = 90;  
const userName = "Alex";  
var age = 20;
```

29. Question: What are JavaScript data types?

Answer: JavaScript data types include strings, numbers, booleans, undefined, null, BigInt, symbols, and objects.

Simple Explanation: Data types describe the kinds of values a program can work with.

Example:

```
let name = "Alex";  
let score = 95;  
let passed = true;  
let result = null;
```

30. Question: What are JavaScript operators?

Answer: Operators perform actions such as arithmetic, comparison, assignment, and logical evaluation.

Simple Explanation: They help calculate values and make decisions in a program.

Example:

```
let total = 10 + 20;  
let isPassed = total >= 25;
```

31. Question: What are conditional statements in JavaScript?

Answer: Conditional statements execute different code depending on whether a condition is true or false.

Simple Explanation: The if, else if, and else statements allow a program to make decisions.

Example:

```
let score = 75;
if (score >= 50) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

32. Question: What are loops in JavaScript?

Answer: Loops repeat a block of code while a specified condition or iteration rule allows it.

Simple Explanation: Loops are useful when performing the same operation on several items.

Example:

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

33. Question: What is a function in JavaScript?

Answer: A function is a reusable block of code designed to perform a particular task.

Simple Explanation: Functions reduce repeated code and make programs easier to organize.

Example:

```
function greet(name) {
  return "Hello, " + name;
}
console.log(greet("Alex"));
```

34. Question: What are function parameters and return values?

Answer: Parameters receive input values, while a return value sends a result back to the code that called the function.

Simple Explanation: Parameters allow a function to work with different inputs, and return provides its output.

Example:

```
function add(a, b) {  
    return a + b;  
}  
let result = add(5, 3);
```

35. Question: What is an array in JavaScript?

Answer: An array is an object used to store an ordered collection of values.

Simple Explanation: Arrays let developers keep related data in one variable and access individual items by index.

Example:

```
let skills = ["HTML", "CSS", "JavaScript"];  
console.log(skills[0]);
```

36. Question: What is an object in JavaScript?

Answer: An object stores related information as key-value properties.

Simple Explanation: Objects are useful for representing entities such as users, products, and assessment results.

Example:

```
let user = {  
    name: "Alex",  
    score: 85  
};  
console.log(user.name);
```

37. Question: What are JavaScript events?

Answer: Events are actions or occurrences that JavaScript can detect and respond to.

Simple Explanation: Events include button clicks, keyboard input, form submissions, and page loading.

Example:

```
<button onclick="showMessage()">Click Me</button>  
<script>
```

```
function showMessage() {  
    alert("Button clicked!");  
}  
</script>
```

38. Question: What is error handling in JavaScript?

Answer: Error handling is the process of detecting and managing errors that occur during program execution.

Simple Explanation: The try, catch, and finally statements help control what happens when an operation fails.

Example:

```
try {  
    JSON.parse("{invalid}");  
} catch (error) {  
    console.log("Invalid data format");  
}
```

39. Question: What are Promises in JavaScript?

Answer: A Promise represents the eventual success or failure of an asynchronous operation.

Simple Explanation: Promises help manage tasks such as retrieving data from a server without blocking the rest of the program.

Example:

```
fetch("https://jsonplaceholder.typicode.com/posts/1")  
    .then(response => response.json())  
    .then(data => console.log(data))  
    .catch(error => console.log(error));
```

Topic 4: Responsive Web Design

40. Question: What is responsive web design?

Answer: Responsive web design makes a website adapt to different screen sizes and devices.

Simple Explanation: A responsive website remains usable on mobile phones, tablets, laptops, and desktop computers.

Example:

```
.container {  
  width: 90%;  
  max-width: 1200px;  
  margin: auto;  
}
```

41. Question: Why is the viewport meta tag important?

Answer: The viewport meta tag helps browsers control a webpage's layout and scaling on mobile devices.

Simple Explanation: Without an appropriate viewport setting, a mobile browser may display a page as if it were designed for a wider screen.

Example:

```
<meta name="viewport"  
  content="width=device-width, initial-scale=1.0">
```

42. Question: What are CSS media queries?

Answer: Media queries apply CSS rules when specified device or viewport conditions are met.

Simple Explanation: They let developers change layouts, font sizes, and spacing for different screen widths.

Example:

```
@media (max-width: 600px) {  
  h1 {  
    font-size: 24px;  
  }  
}
```

43. Question: What is the difference between fixed and fluid layouts?

Answer: A fixed layout uses dimensions that remain constant, while a fluid layout uses flexible dimensions that adapt to available space.

Simple Explanation: Fluid layouts are often more suitable for responsive websites because they adjust to different screen widths.

Example:

```
.fixed {  
  width: 400px;
```

```
}  
.fluid {  
  width: 80%;  
}
```

44. Question: What are relative CSS units?

Answer: Relative CSS units define sizes in relation to another value, such as a parent element, font size, or viewport.

Simple Explanation: Units such as %, em, rem, vw, and vh help create flexible designs.

Example:

```
.container {  
  width: 80%;  
  font-size: 1rem;  
}
```

45. Question: What is mobile-first design?

Answer: Mobile-first design begins with the small-screen experience and progressively enhances the layout for larger screens.

Simple Explanation: This approach encourages developers to prioritize essential content and functionality before adding larger-screen enhancements.

Example:

```
.cards {  
  display: grid;  
  grid-template-columns: 1fr;  
}  
  
@media (min-width: 768px) {  
  .cards {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}
```

46. Question: How can images be made responsive?

Answer: Images can be made responsive by limiting their width to the available space and preserving their aspect ratio.

Simple Explanation: This prevents images from overflowing their containers on small screens.

Example:

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

47. Question: How can Flexbox help create responsive layouts?

Answer: Flexbox arranges items flexibly and can wrap them when space becomes limited.

Simple Explanation: It is useful for navigation bars, button groups, and card layouts that must adapt to different screen sizes.

Example:

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 16px;  
}
```

48. Question: How can CSS Grid help create responsive layouts?

Answer: CSS Grid organizes content into rows and columns that can adapt to available space.

Simple Explanation: Grid can automatically fit cards into as many columns as the container can accommodate.

Example:

```
.cards {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit, minmax(220px, 1fr));  
  gap: 20px;  
}
```

49. Question: What are breakpoints in responsive design?

Answer: Breakpoints are viewport sizes at which a design changes to suit a different available space.

Simple Explanation: Developers choose breakpoints based on where their content or layout needs adjustment, rather than relying only on specific device names.

Example:

```
@media (max-width: 768px) {  
  nav {  
    flex-direction: column;  
  }  
}
```

50. Question: Why is responsive navigation important?

Answer: Responsive navigation keeps website links accessible and usable across different screen sizes.

Simple Explanation: On smaller screens, navigation may use a compact menu or vertically arranged links to avoid crowding.

Example:

```
nav {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 12px;  
}
```

51. Question: How can a developer test a responsive website?

Answer: A developer can test a responsive website using browser developer tools and actual devices with different screen sizes.

Simple Explanation: Testing helps identify overflowing content, unreadable text, overlapping elements, and difficult navigation.

Example:

```
body {  
  overflow-wrap: anywhere;  
}
```

Topic 5: DOM Manipulation

52. Question: What is the DOM in web development?

Answer: DOM stands for Document Object Model. It represents an HTML document as a tree of objects that scripts can access and modify.

Simple Explanation: JavaScript uses the DOM to find webpage elements, change their content, update styles, and respond to user actions.

Example:

```
document.title = "Skill Sense";
```

53. Question: How can JavaScript select an element by its ID?

Answer: The `getElementById()` method selects the element with a specified ID.

Simple Explanation: It is useful when JavaScript needs to access one uniquely identified element.

Example:

```
const heading = document.getElementById("title");
```

54. Question: What is the purpose of `querySelector()`?

Answer: `querySelector()` returns the first element matching a specified CSS selector.

Simple Explanation: It can select elements using IDs, classes, tags, and more complex CSS selectors.

Example:

```
const button = document.querySelector(".start-btn");
```

55. Question: What is the difference between `querySelector()` and `querySelectorAll()`?

Answer: `querySelector()` returns the first matching element, while `querySelectorAll()` returns a static `NodeList` of all matching elements.

Simple Explanation: Use the first method for one element and the second when working with several matching elements.

Example:

```
const firstCard = document.querySelector(".card");  
const allCards = document.querySelectorAll(".card");
```

56. Question: How can JavaScript change the text of an HTML element?

Answer: JavaScript can use the `textContent` property to change an element's text.

Simple Explanation: This allows webpage messages and headings to update dynamically.

Example:

```
document.getElementById("title").textContent = "Welcome to Skill Sense";
```

57. Question: How can JavaScript change CSS styles dynamically?

Answer: JavaScript can modify an element's style property to apply inline CSS changes.

Simple Explanation: This is useful for changing colors, visibility, and other visual properties in response to user actions.

Example:

```
const heading = document.getElementById("title");  
heading.style.color = "purple";
```

58. Question: How can JavaScript add or remove a CSS class?

Answer: JavaScript can use the `classList.add()`, `classList.remove()`, and `classList.toggle()` methods.

Simple Explanation: These methods allow elements to switch between different visual states without manually rewriting their styles.

Example:

```
const card = document.querySelector(".card");  
card.classList.add("active");
```

59. Question: What is event handling in DOM manipulation?

Answer: Event handling allows JavaScript to run code when a particular event occurs on an element.

Simple Explanation: Developers can respond to actions such as clicks, typing, and form submissions.

Example:

```
document.getElementById("startBtn")  
  .addEventListener("click", function () {  
    alert("Assessment started");  
  });
```

60. Question: How can JavaScript create a new HTML element?

Answer: JavaScript can create an element using `document.createElement()`.

Simple Explanation: New elements can be generated dynamically when a user adds an item or receives new content.

Example:

```
const paragraph = document.createElement("p");  
paragraph.textContent = "Welcome!";  
document.body.appendChild(paragraph);
```

61. Question: How can JavaScript remove an HTML element?

Answer: JavaScript can remove an element using its `remove()` method.

Simple Explanation: Removing elements helps update a page when content is no longer needed.

Example:

```
const message = document.getElementById("message");  
message.remove();
```

62. Question: What is the difference between `innerHTML` and `textContent`?

Answer: `innerHTML` reads or sets HTML markup, while `textContent` reads or sets text content.

Simple Explanation: Use `textContent` for ordinary text. Inserting untrusted content through `innerHTML` can introduce cross-site scripting vulnerabilities.

Example:

```
document.getElementById("message").textContent = "Assessment completed";
```

63. Question: What is event delegation in JavaScript?

Answer: Event delegation attaches an event listener to a parent element to handle events from its child elements.

Simple Explanation: It can reduce the number of event listeners and handle dynamically added child elements.

Example:

```
document.getElementById("options")  
  .addEventListener("click", function (event) {  
    if (event.target.matches("button")) {  
      console.log(event.target.textContent);  
    }  
  });
```

});

Topic 6: Forms & Validation

Q64. What is HTML form validation?

Answer: HTML form validation is the process of checking whether the data entered by a user is valid before submitting the form.

Simple Explanation: It helps ensure that users enter the required and correct type of information.

Example: Using required to make sure a name field is not left empty.

Q65. What is the purpose of the required attribute in HTML forms?

Answer: The required attribute makes a form field mandatory before the form can be submitted.

Simple Explanation: The user must enter a value in that field.

Example: `<input type="text" required>` makes the text field compulsory.

Q66. What is the purpose of the placeholder attribute in HTML forms?

Answer: The placeholder attribute provides a short hint that describes the expected input in a form field.

Simple Explanation: It helps users understand what information they should enter.

Example: `<input type="text" placeholder="Enter your name">`

Q67. What is the pattern attribute in HTML forms?

Answer: The pattern attribute specifies a regular expression that the input value must match.

Simple Explanation: It is used to control the format of data entered by the user.

Example: `<input type="text" pattern="[A-Za-z]+">`

Q68. What is client-side validation?

Answer: Client-side validation checks user input in the browser before the data is sent to the server.

Simple Explanation: It helps identify incorrect or missing data immediately.

Example: Checking whether an email field contains a valid email format.

Q69. What is server-side validation?

Answer: Server-side validation checks the submitted data on the server before processing or storing it.

Simple Explanation: It provides an additional layer of validation and security.

Example: A server checks whether a submitted username is valid before saving it.

Q70. What is the type="email" attribute used for?

Answer: The type="email" attribute is used to create an input field that expects a valid email address.

Simple Explanation: The browser can automatically check whether the entered value looks like an email address.

Example: `<input type="email" placeholder="Enter your email">`

Q71. What is the minlength attribute in HTML?

Answer: The minlength attribute specifies the minimum number of characters allowed in an input field.

Simple Explanation: It prevents users from entering very short values.

Example: `<input type="text" minlength="5">`

Q72. What is the maxlength attribute in HTML?

Answer: The maxlength attribute specifies the maximum number of characters allowed in an input field.

Simple Explanation: It limits how many characters a user can enter.

Example: `<input type="text" maxlength="20">`

Q73. What is the min attribute used for?

Answer: The min attribute specifies the minimum allowed value for an input field.

Simple Explanation: It prevents values smaller than the specified limit.

Example: `<input type="number" min="1">`

Q74. What is the max attribute used for?

Answer: The max attribute specifies the maximum allowed value for an input field.

Simple Explanation: It prevents users from entering values greater than the specified limit.

Example: `<input type="number" max="100">`

Q75. Why is form validation important in web development?

Answer: Form validation ensures that users provide correct and complete information before the data is processed.

Simple Explanation: It improves data quality, user experience, and security.

Example: Checking that a registration form contains a valid email and password.

Topic 7: Web APIs & Browser Storage

Q76. What is a Web API?

Answer: A Web API is a set of rules and interfaces that allows web applications to communicate with browser features or external services.

Simple Explanation: APIs help websites access useful functionality and data.

Example: The Geolocation API can be used to access a user's location with permission.

Q77. What is the Fetch API in JavaScript?

Answer: The Fetch API is used to make HTTP requests and retrieve resources from servers.

Simple Explanation: It allows JavaScript to communicate with web servers.

Example: `fetch("https://example.com/data")`

Q78. What is JSON?

Answer: JSON stands for JavaScript Object Notation and is a lightweight format used for storing and exchanging data.

Simple Explanation: It represents data in a simple key-value structure.

Example: `{"name":"John","age":20}`

Q79. What is localStorage?

Answer: localStorage is a browser storage mechanism that allows websites to store data on the user's device.

Simple Explanation: Stored data remains available even after the browser is

closed.

Example: `localStorage.setItem("name", "John");`

Q80. What is sessionStorage?

Answer: sessionStorage stores data for the duration of a browser tab session.

Simple Explanation: The stored data is removed when the tab or window is closed.

Example: `sessionStorage.setItem("theme", "dark");`

Q81. What is the difference between localStorage and sessionStorage?

Answer: localStorage keeps data until it is manually removed, while sessionStorage keeps data only during the current browser session.

Simple Explanation: They differ mainly in how long the stored data remains available.

Example: User preferences can be stored in localStorage, while temporary form data can be stored in sessionStorage.

Q82. What is the purpose of cookies in web development?

Answer: Cookies are small pieces of data stored by the browser and used to remember information between requests.

Simple Explanation: They can help websites remember users and preferences.

Example: A website can use cookies to maintain a login session.

Q83. What is an HTTP request?

Answer: An HTTP request is a message sent by a client to a server to request or submit information.

Simple Explanation: It allows browsers and servers to communicate.

Example: A browser sends a GET request to retrieve a webpage.

Q84. What is an HTTP response?

Answer: An HTTP response is the message returned by a server after processing an HTTP request.

Simple Explanation: It contains the requested data or information about the result of the request.

Example: A server may return an HTML page with a 200 OK status.

Q85. What is an API endpoint?

Answer: An API endpoint is a specific URL through which an application can access a particular resource or service.

Simple Explanation: It acts as an access point for requesting data from an API.

Example: /api/users can be an endpoint for retrieving user information.

Q86. What is the purpose of the Geolocation API?

Answer: The Geolocation API allows a web application to request the geographical location of a user.

Simple Explanation: It can provide location information when the user gives permission.

Example: A map website can use it to show the user's current position.

Q87. What is asynchronous programming in JavaScript?

Answer: Asynchronous programming allows JavaScript to perform tasks without blocking the execution of other operations.

Simple Explanation: Long-running tasks can continue while other code remains responsive.

Example: Fetching data from a server without freezing the webpage.

Q88. What is a Promise in JavaScript?

Answer: A Promise is an object that represents the eventual completion or failure of an asynchronous operation.

Simple Explanation: It helps handle operations that finish later.

Example: A fetch() request returns a Promise.

Topic 8: Web Security & Web Development Basics**Q89. What is web security?**

Answer: Web security refers to protecting websites and web applications from unauthorized access, attacks, and data loss.

Simple Explanation: It helps keep websites and user information safe.

Example: Using HTTPS to protect data transferred between a browser and server.

Q90. What is HTTPS?

Answer: HTTPS stands for HyperText Transfer Protocol Secure and provides encrypted communication between a browser and a web server.

Simple Explanation: It helps protect sensitive information during transmission.

Example: Online banking websites commonly use HTTPS.

Q91. What is XSS?

Answer: XSS, or Cross-Site Scripting, is a security attack in which malicious scripts are injected into webpages viewed by users.

Simple Explanation: It can allow attackers to execute unwanted scripts in another user's browser.

Example: A website that displays untrusted user input without proper sanitization may be vulnerable to XSS.

Q92. What is SQL Injection?

Answer: SQL Injection is an attack in which malicious SQL statements are inserted into application inputs to manipulate a database.

Simple Explanation: It can expose, modify, or delete database information.

Example: An insecure login form may allow an attacker to manipulate a database query.

Q93. What is authentication?

Answer: Authentication is the process of verifying the identity of a user.

Simple Explanation: It checks whether a person is really who they claim to be.

Example: Logging into a website using a username and password.

Q94. What is authorization?

Answer: Authorization is the process of determining what resources or actions an authenticated user is allowed to access.

Simple Explanation: It decides what a user can do after logging in.

Example: An administrator can access a dashboard that a normal user cannot.

Q95. What is responsive web design?

Answer: Responsive web design is an approach that makes websites adapt to different screen sizes and devices.

Simple Explanation: The website should work properly on desktops, tablets, and smartphones.

Example: A navigation menu changing layout on a mobile screen.

Q96. What is a media query in CSS?

Answer: A media query is a CSS feature used to apply different styles based on device or screen conditions.

Simple Explanation: It helps create responsive websites.

Example: @media (max-width: 768px) { ... }

Q97. What is website accessibility?

Answer: Website accessibility means designing websites so that people with different abilities can use them effectively.

Simple Explanation: It makes web content usable by a wider range of users.

Example: Adding meaningful alt text to images for screen readers.

Q98. What is SEO in web development?

Answer: SEO stands for Search Engine Optimization and involves improving a website so that it can be discovered and ranked better by search engines.

Simple Explanation: Good SEO can help more users find a website through search engines.

Example: Using meaningful page titles and relevant content.

Q99. What is website performance optimization?

Answer: Website performance optimization is the process of improving a website's loading speed and overall responsiveness.

Simple Explanation: Faster websites provide a better user experience.

Example: Compressing images to reduce page loading time.

Q100. What is the role of a web browser in web development?

Answer: A web browser interprets and displays web technologies such as HTML, CSS, and JavaScript to users.

Simple Explanation: It allows users to view and interact with web applications.

Example: Chrome loads an HTML page, applies CSS styles, and executes JavaScript.

